

Système de stockage distribué basé sur une DHT

Projet Master Systèmes et Applications Répartis (SAR)

PSAR – Sorbonne Université

Auteurs: Cedric Genevieve, Safa Gunes, Somaya Boudalil
Étudiants en Master 2026 — Systèmes et Applications Répartis

Instructeur: Prof. Alessio Pagliari

Date: 14 avril 2026

Table des matières

1	Analyse Comparative des Paradigmes Architecturaux de Stockage Distribué	3
1.1	Étude des approches P2P structurée: Les DHTs	3
1.1.1	Introduction au problème du Lookup	3
1.1.2	Comparaison des stratégies de recherche	4
1.1.3	Fondamentaux des Distributed Hash Tables	6
1.1.4	Mécanismes des DHTs	7
1.1.5	Interfaces des DHTs	8
1.1.6	Conclusions	8
1.2	Étude de l'approche hiérarchisée : HBase	9
1.2.1	Fondamentaux architecturaux de HBase	9
1.2.2	Mécanismes internes : partitionnement et gestion des pannes	11
1.2.3	Avantages et limites	12
1.3	Conclusion et justification du choix architectural	12
2	Méthode de répartition des données	13
2.1	Motivation et définition de la répartition des données	13
2.2	Les différentes méthodes de répartition des données	13
2.2.1	hachage classique	14
2.2.2	Hachage cohérent (Consistent Hashing)	15
2.3	Conclusion sur les méthodes de répartition des données	19
3	Approche naïve : une DHT non structurée inspirée de Gnutella	20
3.1	Positionnement général et lien avec Gnutella	20
3.2	Problème initial : l'absence de vue globale	20
3.3	Passage à un mesh : ce que cela corrige, et ce que cela casse	21
3.4	Le flooding comme correction partielle	23
3.5	Réponse directe, protocole et concurrence	24
3.6	Découverte et maintien de la vivacité	25
3.7	Usages réels et intérêt pratique	25
3.8	Bilan	26
4	Chord	26
4.1	Principe de fonctionnement	26
4.2	Routage efficace : Finger Table	26
4.3	Algorithmes essentiels	27
4.4	Complexité (résumé)	27
5	Pastry	27
5.1	Architecture générale	27
5.2	Identifiants et espace de clés	28
5.3	Structures de routage	28
5.3.1	1. Routing Table	28
5.3.2	2. Leaf Set (ensemble des feuilles)	28

5.3.3	3. Neighbor Set (ensemble des voisins)	28
5.4	Processus de routage	28
5.5	Algorithmes des nœuds et maintenance	29
6	Architecture du système et cycle de vie d'une requête	30
6.1	Choix architecturaux et problématique	30
6.2	Architecture générale basée sur Chord	30
6.2.1	Couche Overlay et Couche applicative	30
6.2.2	Cycle de vie d'une requête	30
6.2.3	Conclusion sur l'architecture Chord	31
6.3	Architecture générale basée sur Pastry	31
6.3.1	Couche overlay et couche applicative dans Pastry	31
6.3.2	Cycle de vie d'une requête dans Pastry	32
6.3.3	Conclusion sur l'architecture Pastry	32

1 Analyse Comparative des Paradigmes Architecturaux de Stockage Distribué

La croissance du volume de données a rendu les architectures centralisées insuffisantes. Les systèmes distribués sont devenus la norme, mais posent un défi central : garantir un *lookup* efficace, la *consistency* et la *fault tolerance* dans un environnement de nœuds peu fiables.

Dans ce projet Master, nous comparons deux architectures de référence pour le stockage distribué, basées sur des paradigmes différents :

1. L'architecture P2P structurée (Les DHTs) Paradigme décentralisé (architecture *symmetric*) : les nœuds ont un rôle homogène et s'auto-organisent dans un *overlay* structuré. Nous considérons plusieurs DHT (Chord, CAN, Pastry, Kademlia) qui résolvent le *lookup* en général en $O(\log N)$, avec des compromis différents entre topologie logique, latence de routage, taille des tables d'état et résilience au *churn*.

2. L'architecture hiérarchisée orientée colonnes (Modèle HBase) À l'opposé du P2P pur, ce paradigme suit une architecture *master-worker* inspirée de Bigtable. HBase vise une cohérence forte, un modèle orienté colonnes/versionné, et conserve une centralisation partielle (HMaster) pour la gestion des métadonnées et des transactions, en échange de meilleures performances d'écriture et de scan. La suite du document présente d'abord le routage et la maintenance des DHTs, puis l'architecture interne de HBase et ses mécanismes de persistance.

1.1 Étude des approches P2P structurée: Les DHTs

Cette section résume les principes essentiels des *Distributed Hash Tables* (DHTs) : localisation des données, routage, gestion des nœuds et interfaces applicatives.

Référence : WEHRLE, GÖTZ et RIECHE (2005).

1.1.1 Introduction au problème du Lookup

Le problème central est le *lookup problem* : où stocker une donnée D , puis comment la retrouver dans un réseau distribué sans coordination centrale.

"Où stocker, et comment trouver un certain élément de donnée D dans un système distribué sans aucun contrôle ou coordination centralisés ?"

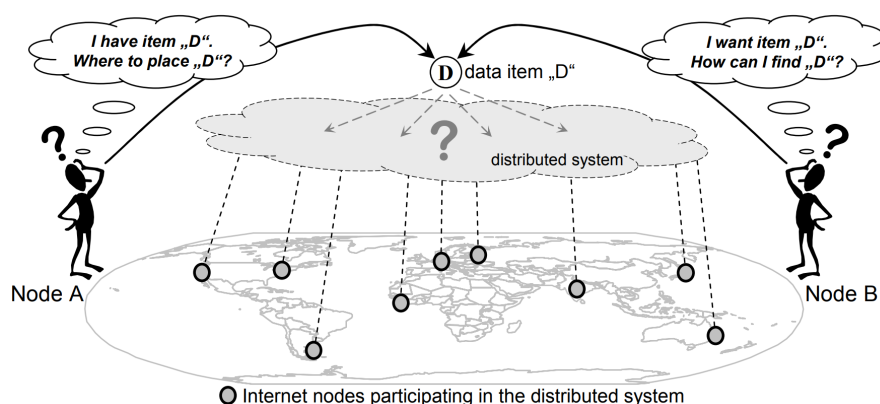


FIGURE 1 – **Le problème du lookup** : le nœud A stocke une donnée D et le nœud B doit la retrouver sans connaître son emplacement. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

1.1.2 Comparaison des stratégies de recherche

Trois familles de solutions sont comparées.

1. Central Server

Approche de **première génération** (ex. Napster). Un serveur central indexe les contenus. Recherche rapide en $O(1)$, mais dépendance forte à un *single point of failure*.

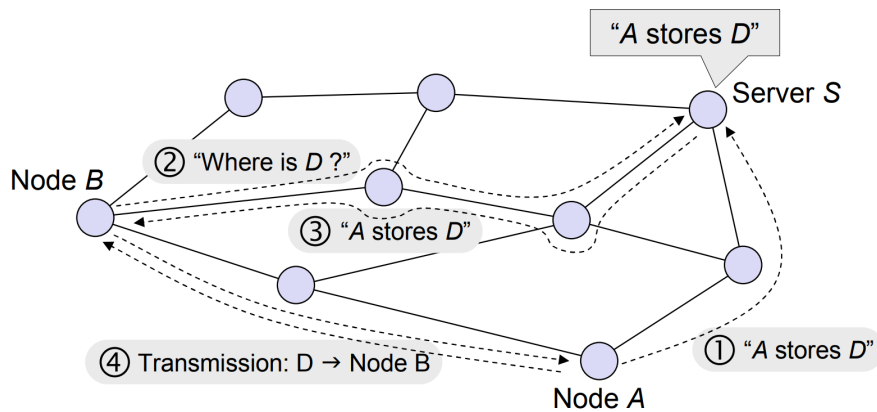


FIGURE 2 – **Central Server** : publication, requête au serveur central, puis téléchargement direct entre pairs. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

2. Flooding Search

Approche de **deuxième génération** (ex. Gnutella). La requête est propagée de voisin en voisin via un *reactive flooding* jusqu'au *Time To Live* (TTL). L'approche évite le serveur central, mais génère un fort *communication overhead* et des résultats non garantis.

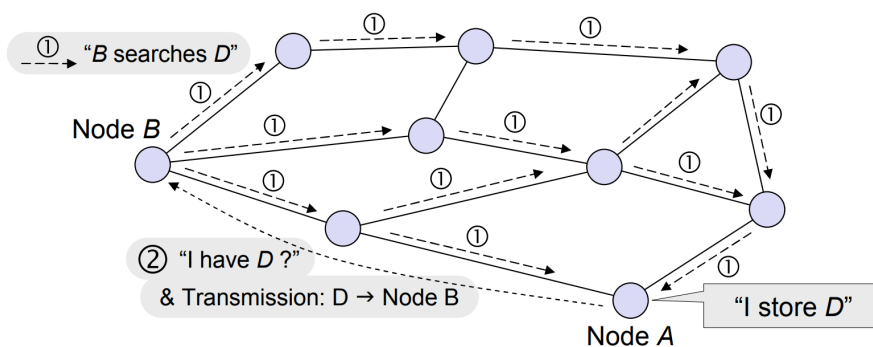


FIGURE 3 – **Flooding Search** : diffusion récursive de requêtes, puis retour des nœuds ayant la donnée. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

3. Distributed Hash Tables (DHTs)

Les DHTs (*structured Peer-to-Peer systems*) structurent l'*overlay* pour obtenir un compromis robuste : $O(\log N)$ entrées de routage par nœud et $O(\log N)$ *hops* pour localiser une donnée.

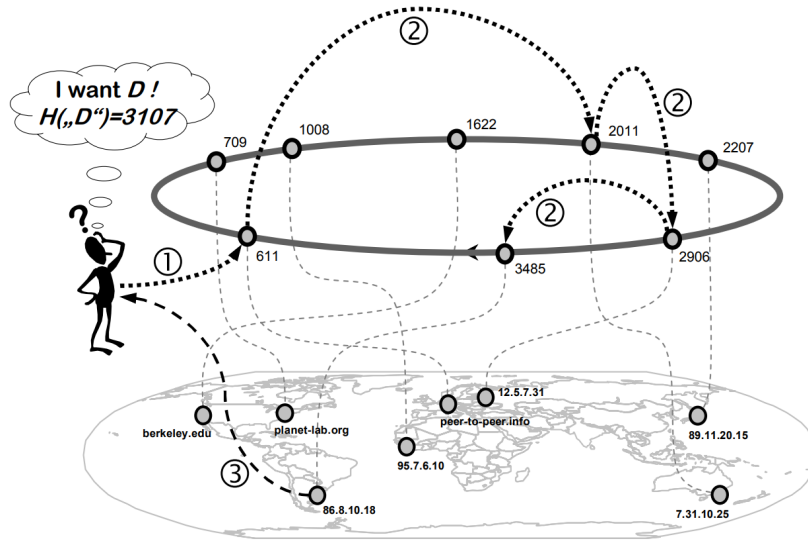


FIGURE 4 – **Distributed Hash Table** : routage structuré et efficace vers le nœud responsable. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

Les Figures 2, 3, 4 et 5 montrent que les DHTs offrent le meilleur compromis entre coût de recherche et coût de stockage.

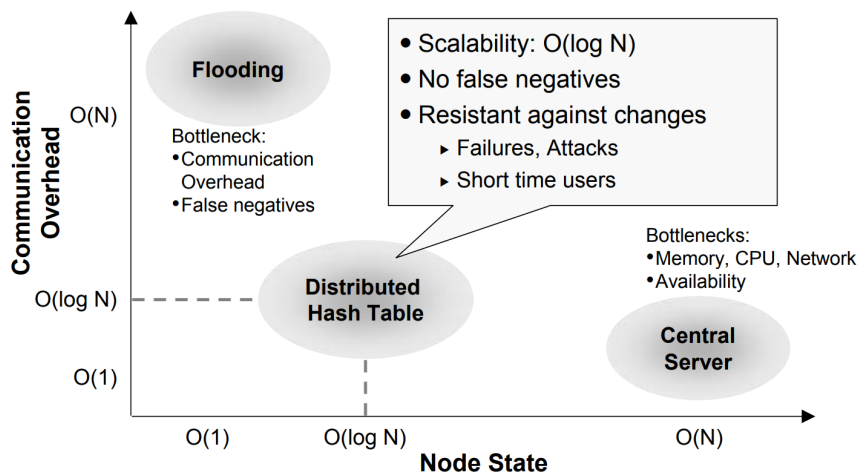


FIGURE 5 – Comparaison de la complexité en termes d'effort de recherche (axe y) et de coût de stockage par nœud (axe x). Les goulots d'étranglement (*bottlenecks*) ainsi que les caractéristiques spécifiques de chaque approche sont indiqués. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

1.1.3 Fondamentaux des Distributed Hash Tables

Une DHT associe chaque clé à un nœud responsable via un espace d'adressage logique et un routage déterministe.

Gestion distribuée des données

Chaque nœud gère une plage de clés et conserve une *partial view* du système. Les messages sont relayés de nœud en nœud jusqu'à la destination.

Adressage dans les Distributed Hash Tables

Nœuds et données partagent le même *address space* logique (souvent très grand, p. ex. 0 à $2^{160} - 1$). Les IDs sont produits par une *collision-resistant hash function*.

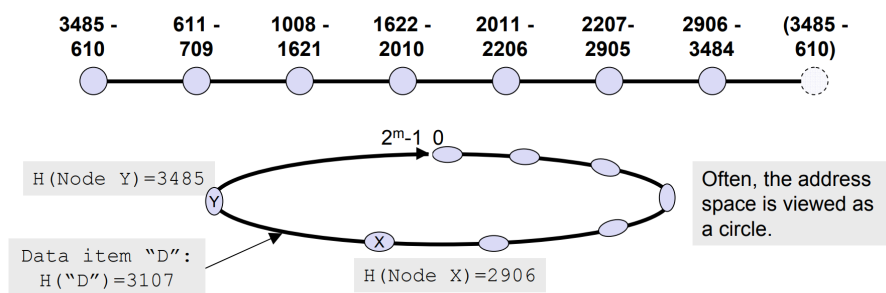


FIGURE 6 – Espace d'adressage linéaire partitionné entre paires. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

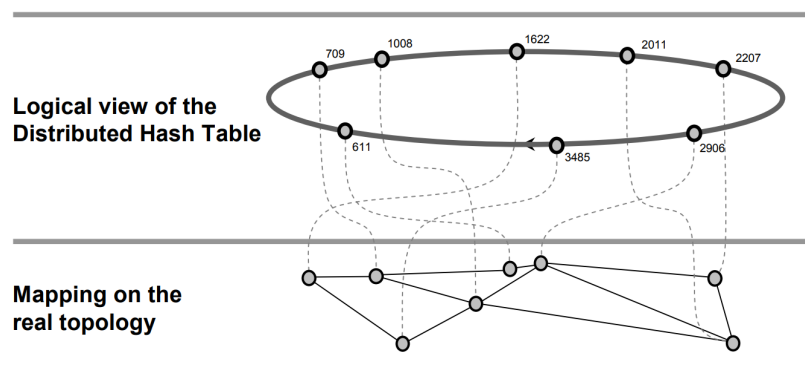


FIGURE 7 – Vue logique de la DHT et vue du réseau sous-jacent. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

Routage

Chaque nœud transmet vers un voisin plus proche de l'ID cible, ce qui réduit progressivement la distance jusqu'au nœud responsable en un *small number of hops*.

Méthodes de stockage

Direct Storage : la donnée est copiée sur le nœud responsable (meilleure *availability*, coût plus élevé).

Indirect Storage : la DHT stocke un *pointer* vers la donnée (coût réduit, dépendance au nœud source).

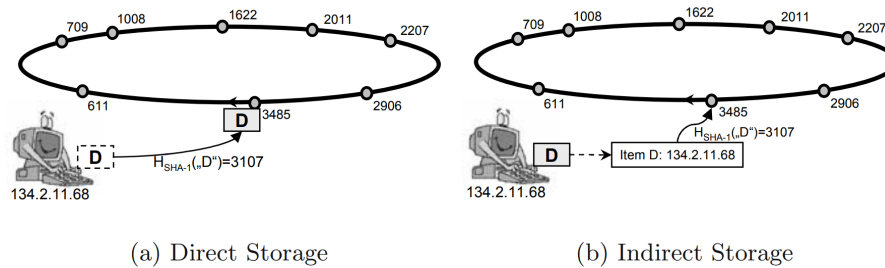


FIGURE 8 – Comparaison entre stockage direct et stockage indirect. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

1.1.4 Mécanismes des DHTs

La robustesse dépend de la gestion du *churn*, c'est-à-dire le taux de *joins* et *leaves* (arrivées/départs) ainsi que les pannes fréquentes.

Arrivée de nœuds

L'entrée d'un nœud suit quatre étapes courtes :

1. **Bootstrap** : le nouveau pair contacte un nœud existant qui sert d'*entry point*.
2. **Address Assignment** : il reçoit sa partition dans le *logical address space*.
3. **Routing Update** : les tables de routage sont ajustées pour intégrer le nouveau pair.
4. **Data Transfer** : les paires (*key*, *value*) devenues sa responsabilité lui sont transférées.

Défaillance de nœuds

Deux approches coexistent : *reactive recovery* (détection pendant le routage, puis route alternative) et *proactive recovery* (*probes* périodiques, remplacement préventif des entrées invalides). La *replication* limite la perte de données applicatives.

Départ de nœuds

Un départ volontaire (*graceful leave*) facilite la migration immédiate des données vers le *successor* et accélère la convergence des tables de routage. Il améliore aussi la *routing efficiency* et stabilise les mécanismes de *replication* et de *load-balancing*.

1.1.5 Interfaces des DHTs

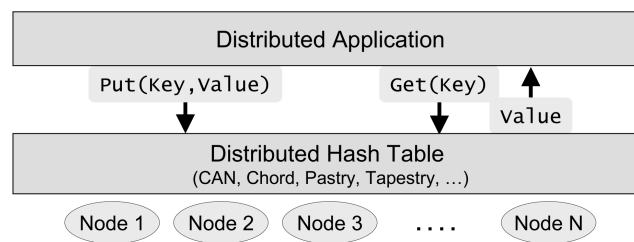


FIGURE 9 – Interface d’une Distributed Hash Table, typiquement put/get. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

Interface de routage

Interface *stateless* orientée messages : *send* livre vers un ID destination, *receive* remet les messages à l’application (*delivery semantics*). Cette *stateless interface* se concentre sur la livraison de paquets basés sur un ID logique plutôt que sur une adresse IP physique.

Interface de stockage

Interface clé-valeur : *put* écrit la donnée, *get* la récupère. La couche gère *routing failures*, *replication* et *admission control*. Cette couche ajoute de la complexité car elle gère la persistance et la fiabilité des données, comme une table de *hash* distribuée.

Interface Client

Des hôtes peuvent consommer le service DHT sans participer au routage de l’*overlay* (mode *client*). Ce mode est utile quand on veut séparer usage applicatif et participation au routage (fiabilité, politiques d’*access control*).

1.1.6 Conclusions

Les DHTs offrent une excellente *scalability* sans point central, mais imposent des compromis entre efficacité de routage, coût de maintenance et résilience au *churn*.

Défis de conception

- **Routing Efficiency** : qualité du routage selon la structure des tables et la topologie.
- **Management Overhead** : coût de maintenance en continu.
- **Churn** : impact des arrivées/départs simultanés sur la stabilité.

Limitations fondamentales

- **Untrusted Environments** : résistance difficile face aux nœuds malveillants et aux *Byzantine faults*.
- **Query Constraints** : *fuzzy searches* et requêtes textuelles complexes peu adaptées au modèle par identifiants.

1.2 Étude de l'approche hiérarchisée : HBase

Les DHT répondent bien au *lookup* (retrouver rapidement la machine qui détient une clé) dans des systèmes où les nœuds entrent et sortent fréquemment. En revanche, certaines charges exigent une cohérence plus stricte (lecture après écriture fiable), une meilleure persistance (données conservées malgré les pannes) et de bonnes performances de scan (lecture de plages de clés) en plus des écritures. L'approche hiérarchisée d'Apache HBase cible précisément ce compromis.

HBase, construit sur HDFS (*Hadoop Distributed File System*, couche de stockage distribué et répliqué), est une base NoSQL distribuée orientée grands volumes, c'est-à-dire une base non relationnelle conçue pour monter horizontalement sur plusieurs serveurs. Contrairement aux DHT décentralisées, HBase adopte un modèle *master-worker* : le Master coordonne le cluster et les workers (Region-Servers) servent les données. Le *range partitioning* découpe les données en plages de clés triées ; chaque *row key* (identifiant de ligne) est donc routée de manière déterministe, ce qui favorise les scans séquentiels et une cohérence forte au niveau ligne.

1.2.1 Fondamentaux architecturaux de HBase

Avant de détailler les opérations PUT/GET, il est utile de situer les rôles des composants principaux. HBase s'appuie sur une architecture en couches : un **plan de contrôle** (HMaster et ZooKeeper) pour l'orchestration, un **plan de données** (RegionServers) pour les accès applicatifs, et une couche de persistance (WAL + HFiles sur HDFS) pour la durabilité.

Dans cette organisation, le client n'interagit pas en continu avec le Master pour chaque requête. Le Master sert surtout à localiser les régions et à gérer le cycle de vie du cluster (allocation, équilibrage, reprise sur panne), tandis que les échanges de lecture/écriture se font ensuite directement avec le RegionServer responsable de la plage de clés visée.

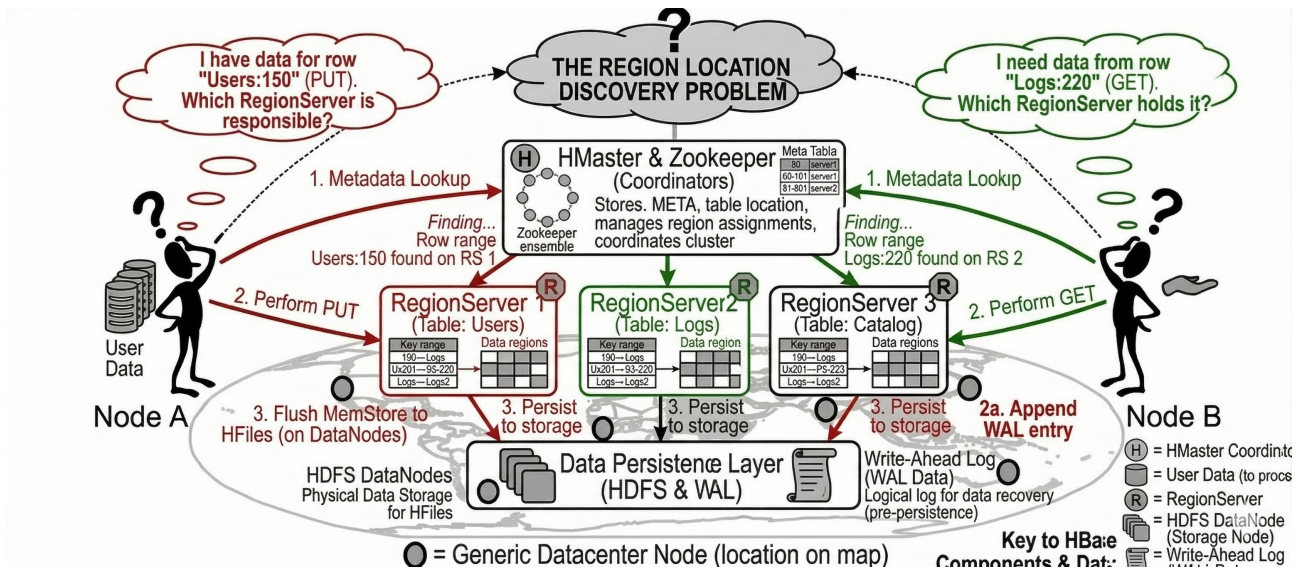


FIGURE 10 – Ce diagramme illustre l'architecture distribuée d'Apache HBase et met en évidence le problème de la découverte de l'emplacement des régions. Il montre comment deux nœuds clients, A et B, interagissent avec le système. Les RegionServers et HMaster sont géographiquement distribués sur la carte, soulignant la nature distribuée du cluster. La première étape, commune, est la recherche de métadonnées (**étape 1**) : les clients interrogent le coordinateur HMaster/ZooKeeper (localisé sur un nœud de datacenter générique) pour trouver le RegionServer responsable de la plage de clés spécifique. Pour le nœud A, effectuant une opération PUT (écriture) pour une ligne d'utilisateur, HMaster localise la plage de clés sur le RegionServer 1 (RS1). Le nœud A effectue ensuite directement le PUT sur RS1 (**étape 2**), puis RS1 vidange son MemStore vers les HFiles sur les DataNodes HDFS (**étape 3**). Pour le nœud B, effectuant une opération GET (lecture) pour une ligne de logs, HMaster localise la plage de clés sur le RegionServer 2 (RS2), puis le nœud B effectue directement le GET sur RS2 (**étape 2**). La couche de persistance, située en bas, détaille le Write-Ahead Log (données WAL) pour la récupération logique avant persistance et les DataNodes HDFS pour le stockage physique des fichiers HFiles.

La figure 10 donne une vue macro des composants et de leurs interactions. Avant de passer à l'analyse détaillée des performances, il est utile d'isoler le cycle d'une requête d'écriture puis de lecture pour comprendre où se situent le coût de localisation, le chemin de persistance et le point de service de la donnée.

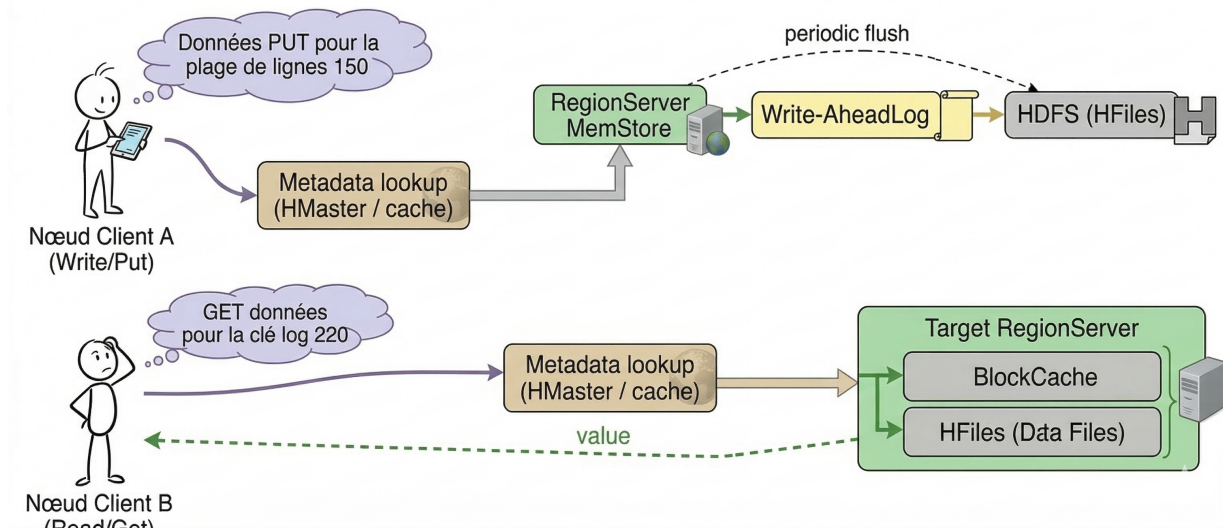


FIGURE 11 – Diagramme simplifié des flux PUT/GET dans HBase. Pour un PUT (client A), le client commence par une résolution de métadonnées (HMaster/cache), envoie l'écriture au RegionServer ciblé (MemStore), puis la donnée est sécurisée via le Write-Ahead Log avant d'être persistée sous forme de HFiles sur HDFS (flush périodique). Pour un GET (client B), le client résout d'abord la localisation de la région, interroge le RegionServer cible, puis la valeur est servie depuis le BlockCache ou les HFiles selon la disponibilité des blocs. Cette vue fournit une lecture opérationnelle concise des chemins d'écriture et de lecture dans HBase.

Cette décomposition met en évidence un point important pour l'analyse comparative avec les DHT : la localisation est semi-indirecte (métadonnées puis accès direct), ce qui réduit le coût de routage après initialisation, mais introduit une dépendance à la qualité de la gestion des régions et du cache client.

Le *HBase Master* gère le cluster (allocation des régions, équilibrage, détection de pannes), tandis que les *RegionServers* stockent les données et exécutent les lectures/écritures. Cette séparation simplifie l'administration et le contrôle opérationnel.

Modèle de données et espace d'adressage

Le modèle est orienté colonnes (tables, *rows*, familles de colonnes). Les *row keys* ordonnées lexico-graphiquement définissent l'espace d'adressage : excellent pour les scans et les requêtes par plages, mais sensible au design des clés.

1.2.2 Mécanismes internes : partitionnement et gestion des pannes

Deux mécanismes portent la scalabilité et la durabilité : partitionnement par régions et réaffectation après panne.

Partitionnement horizontal par plages de clés

Les tables sont découpées en régions couvrant des plages continues de *row keys*. Avec la croissance des données, *region splitting* répartit la charge entre RegionServers. Limite clé : des clés monotones (ex. timestamps) créent des *hotspots* et du *data skew* (*storage skew/access skew*).

Par rapport au *consistent hashing* des DHT (distribution pseudo-aléatoire), HBase choisit un partitionnement déterministe par ordre de clés.

Gestion des pannes et réaffectation

En cas de panne d'un RegionServer, le Master (avec ZooKeeper) réassigne les régions. Avec le *WAL* et HDFS, la reconstruction est robuste. Ce modèle est centralisé, contrairement à la réorganisation incrémentale des DHT.

1.2.3 Avantages et limites

Avantages. Cohérence forte par ligne, scans séquentiels performants et scalabilité horizontale via *region splitting* et ajout de RegionServers.

Limites. Le design des clés reste critique (risques de *hotspot* et de *data skew*), et l'exploitation est plus lourde qu'une DHT pure (HDFS, ZooKeeper, tuning et supervision).

1.3 Conclusion et justification du choix architectural

DHT et HBase répondent au même objectif (stockage distribué à grande échelle) avec des choix opposés : décentralisation + *consistent hashing* d'un côté, coordination centralisée + *range partitioning* de l'autre.

Critère d'évaluation : gestion du data skew

Le *data skew* est déterminant : HBase peut concentrer charge/stockage si les clés sont mal conçues, alors que les DHT réduisent structurellement ce risque via une distribution pseudo-aléatoire des identifiants.

Motivation du choix

Notre objectif est d'implémenter un système totalement décentralisé en anneau logique. Les DHT sont mieux alignées avec cet objectif (routage distribué, auto-organisation, joins/leaves, absence de maître), tandis qu'une approche HBase déplacerait le projet vers l'ingénierie d'un cluster hiérarchisé.

Conclusion du choix

HBase reste excellent pour des besoins de cohérence forte et de scans massifs, mais pour nos objectifs pédagogiques/expérimentaux, l'approche DHT est plus pertinente. En conséquence, nous retenons l'architecture DHT comme base de la suite du projet.

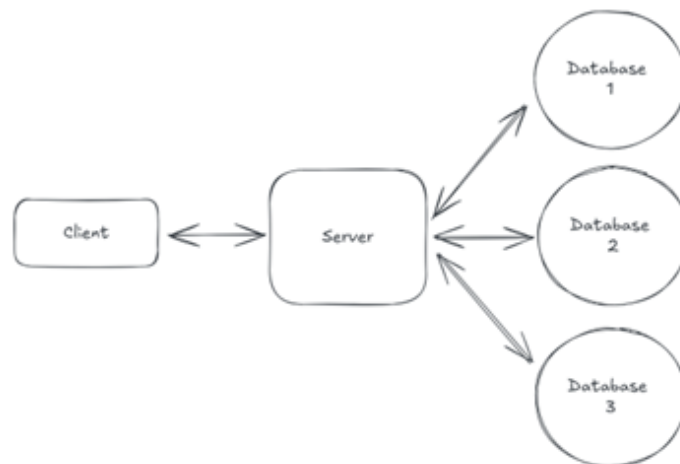
2 Méthode de répartition des données

2.1 Motivation et définition de la répartition des données

Avec l'augmentation continue du volume de données et du nombre d'utilisateurs, les architectures de stockage centralisées atteignent rapidement leurs limites. Une base de données unique n'a plus la capacité de contenir toute ces données et constitue un point de défaillance susceptible de compromettre la disponibilité du système.



Une solution pour pallier à ces limitations consiste à répartir les données et les requêtes sur plusieurs bases de données, afin de distribuer la charge de stockage et de traitement, et d'éviter la surcharge d'une base de données unique.



Toutefois, cette distribution soulève un problème fondamental : il devient nécessaire de déterminer de manière systématique sur quelle base de données une donnée doit être stockée et comment la retrouver efficacement. C'est pour répondre à cette problématique que les méthodes de répartition des données (data partitioning ou data sharding) ont été introduites. Elles définissent les règles de placement des données ce qui impacte directement les performances et l'équilibrage de charge. Ces méthodes de répartitions des données impactent également l'évolution du système dans le temps, c'est-à-dire lors de l'ajout ou le retrait d'une base de données.

2.2 Les différentes méthodes de répartitions des données

Après avoir montré la nécessité de répartir les données dans un système distribué, la première question qu'on se pose : comment associer chaque donnée à une base de données ?

Différentes méthodes de répartition des données permettent d'associer une clé à une base de données dans un système distribué.

Dans les exemples suivant nous allons utiliser le termes nœud qui fait référence à une instance base de données et le terme évènement qui fait référence à une donnée.

2.2.1 hachage classique

Principe général du hachage classique

Une première approche naïve appelée hachage classique (naive hashing ou modulo hashing) consiste à utiliser une fonction de hachage afin de faire la correspondance entre la clé d'une donnée et la base de données dans laquelle nous allons la stocker.

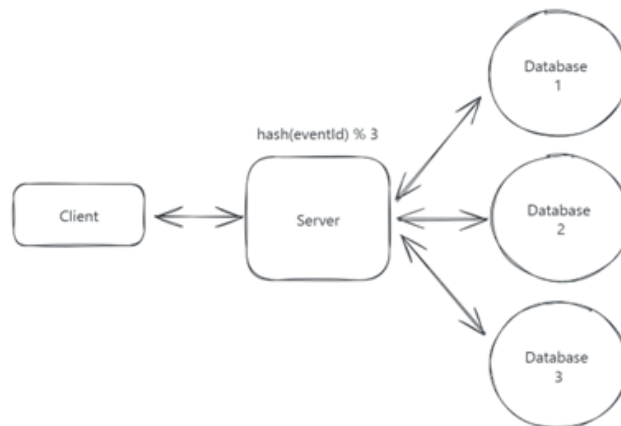
Association d'une clé à un nœud

Cette méthode repose sur les étapes suivantes :

1. Prendre l'id de l'événement et l'exécuter via une fonction de hachage, qui le convertit en nombre
2. Prendre ce nombre et effectuer l'opération modulo (%) avec le nombre de base de données.
3. Le résultat nous indique quelle base de données doit stocker cet événement.

La formule serait donc :

$$\text{database_id} = \text{hash}(\text{event_id}) \bmod \text{number_of_databases}$$



Pour un exemple concret avec 3 bases de données :

Event #1234 → $\text{hash}(1234) \% 3 = 1 \rightarrow \text{Database 2}$

Event #5678 → $\text{hash}(5678) \% 3 = 0 \rightarrow \text{Database 1}$

Event #9012 → $\text{hash}(9012) \% 3 = 2 \rightarrow \text{Database 3}$

Ainsi, chaque clé est associée à une base de données unique, ce qui permet de retrouver l'emplacement d'une donnée à partir de sa clé.

Limite du hachage classique

Le premier problème survient lorsque l'on souhaite ajouter une nouvelle instance de base de données. La première solution à laquelle on pourrait penser consisterait à exécuter l'opération modulo avec 4 au lieu de 3 (Par rapport à l'exemple précédent).

$$\text{database_id} = \text{hash}(\text{event_id}) \% 4$$

La modification du nombre total de bases de données a non seulement affecté les données destinées à la nouvelle instance, mais a également entraîné une modification de la base de données associée à chaque événement précédemment stocké.

En effet, si on reprend un des exemples précédents :

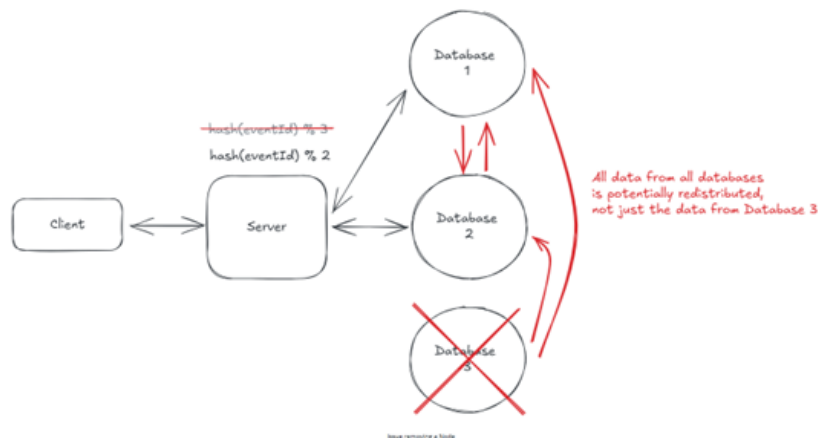
Event #1234 → $\text{hash}(1234) \% 3 = 1 \rightarrow \text{Database 2}$ $\text{hash}(1234) \% 4 = 0 \rightarrow \text{Database 1}$



L'événement n° 1234 était mappé à la base de données 2, mais avec l'ajout de la nouvelle base de données, ces données doivent donc être déplacées vers la base de données 1.

L'ajout d'une ou plusieurs bases de données entraîne donc une redistribution dans toutes les instances de base de données, ce qui provoque un mouvement massif de données inutiles. Ce mouvement massif cause d'énormes pics de charge de base de données et impact directement les utilisateurs. En effet, les utilisateurs ne pourront pas accéder à leur donnée ou auront un temps de réponse lent.

Un autre problème avec le hachage modulo simple réside dans le retrait d'une base de données. Supposons qu'une base de données soit tombée en panne. Il est nécessaire de redistribuer les données qui étaient dans la base de données tombée en panne vers les autres bases de données encore actives. Etant donné que le nombre de base de données actives diminue, le calcul pour trouver où ranger une donnée sera modifié. De ce fait, les données présentes dans les bases de données encore actives risquent d'être redistribuées vers d'autres instances de base de données.



Ces limitations montrent que le hachage modulo classique n'est pas adapté aux systèmes distribués dynamiques, où les bases de données peuvent être ajoutées ou retirées fréquemment.

2.2.2 Hachage cohérent (Consistent Hashing)

Principe général du consistent Hashing

Le consistent Hashing a été introduit pour résoudre le problème de la redistribution des données lors de l'ajout ou de la suppression d'une instance dans un système distribué. Le principe est d'organiser à la fois les données et les bases de données dans un espace circulaire appelé « anneau de hachage ».

Dans cet anneau, les clés et les nœuds sont positionnés en utilisant la même fonction de hachage. Chaque base de données se voit attribuer une position sur l'anneau, et chaque donnée est également

positionnée sur cet anneau en fonction du résultat de sa fonction de hachage.

Association d'une clé à un nœud

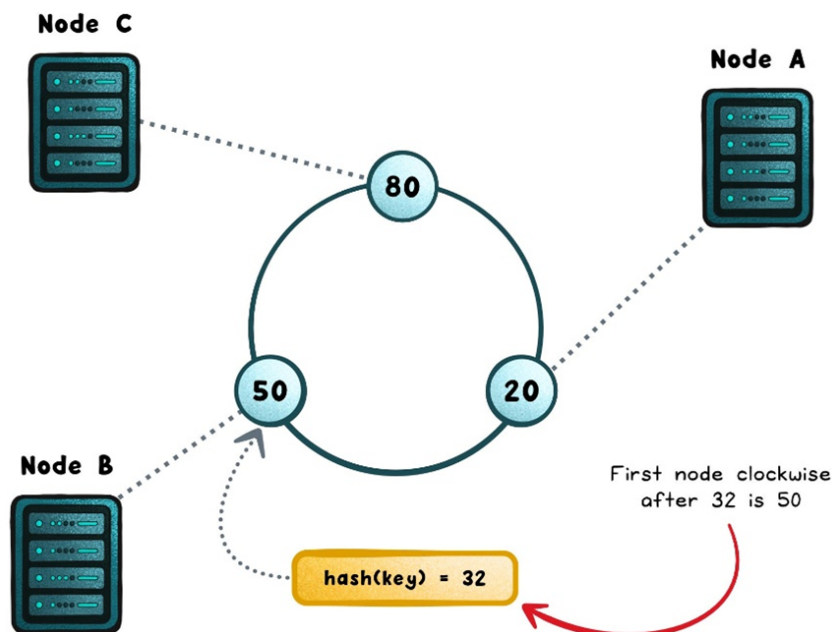
Cette méthode repose sur les étapes suivantes :

1. On réalise d'abord un anneau de hachage avec un nombre fixe de points.
2. On positionne les bases de données sur l'anneau en appliquant la fonction de hachage à leur identifiant.
3. Pour trouver sur quelle base de données une donnée doit être stockée, on hache d'abord l'ID de la donnée comme avec le hachage classique.
4. On se déplace ensuite dans le sens des aiguilles d'une montre sur l'anneau jusqu'à rencontrer le premier nœud.
5. Ce nœud va stocker de la donnée.

Par exemple :

Pour un nombre fixe de points égal à 90, 3 bases de données positionnées aux points 20, 50 et 80.

Si nous avons un $\text{hash}(\text{id_data}) = 32$, nous nous déplaçons dans le sens des aiguilles d'une montre pour trouver la première instance de base de données. Cette instance sera la base de données 2 (Node B) située au point 50.



Ajout d'une nouvelle base de données

Lorsqu'on ajoute une nouvelle base de données, seules les données qui se situent entre sa nouvelle position et la base de données précédente doivent être déplacées.

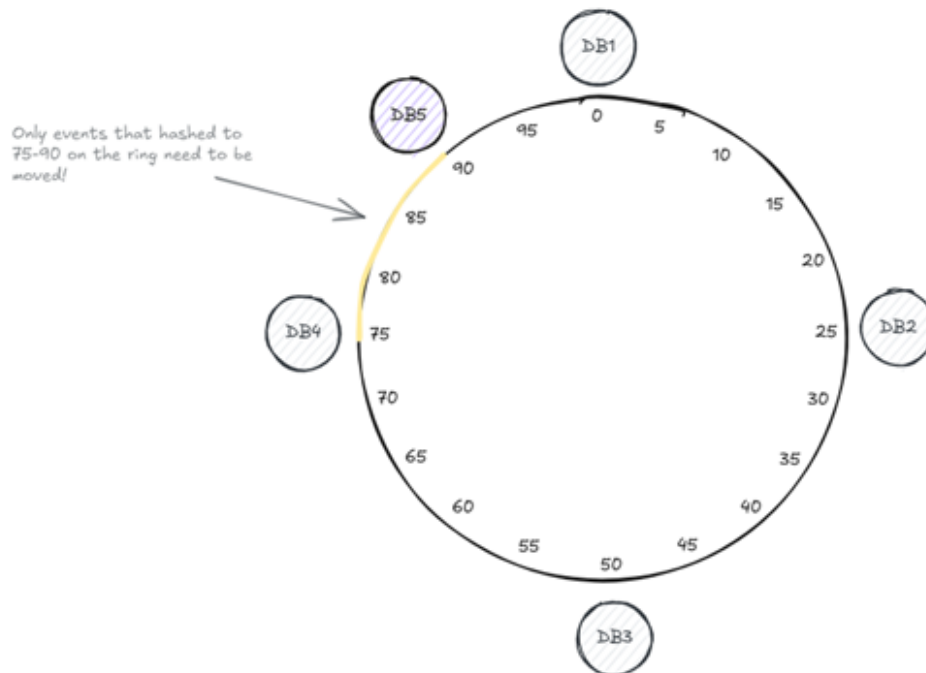
Par exemple :

Pour un nombre fixe de points égal à 100, 4 bases de données positionnées aux points 0, 25, 50 et 75.

Si nous ajoutons une cinquième base de données à la position 90 sur notre anneau :

- Seules les données qui se hachent vers des positions comprises entre 75 et 90 doivent désormais être stockées dans la base de données 5.

- Toutes les autres données restent dans leur base de données respective.



Hash Ring with DB5 added

Contrairement au hachage modulo classique, seule une petite portion des données est redistribuée.

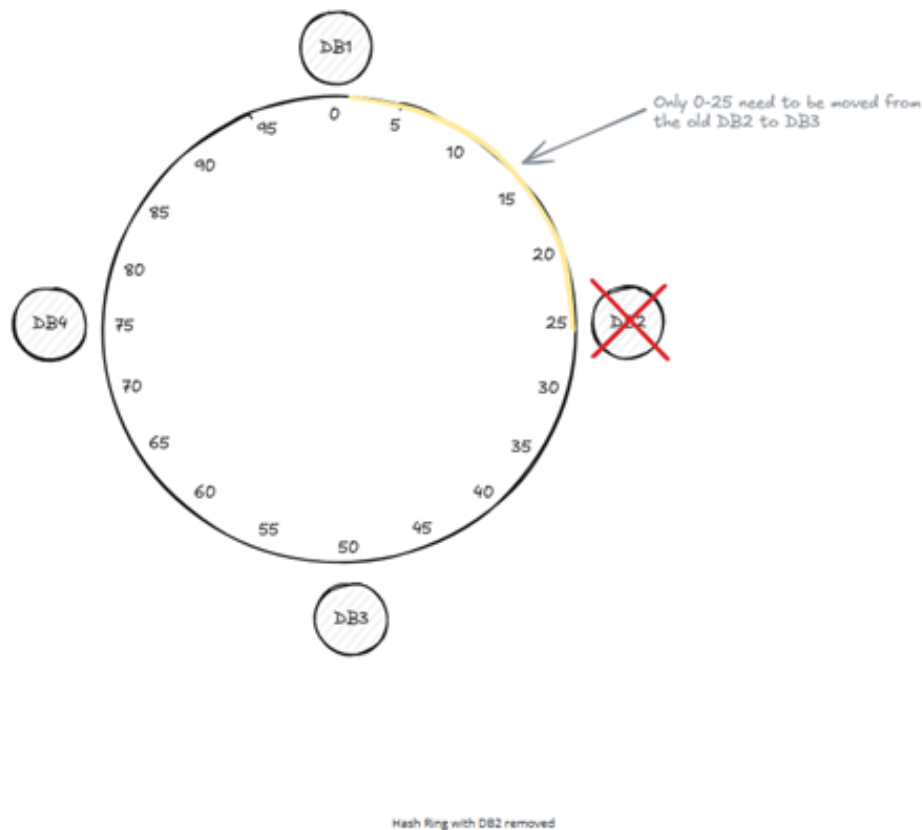
Suppression d'une base de données

En reprenant l'exemple initial :

Pour un nombre fixe de points égal à 100, 4 bases de données positionnées aux points 0, 25, 50 et 75.

Supposons que la base de données numéro 2 située à la position 25 rencontre une défaillance.

- Seules les données mappées sur la base de données 2 doivent être déplacées.
- Le déplacement se fait dans le sens des aiguilles d'une montre. Elles seront donc mappées à la base de données 3 située à la position 50.
- Toutes les autres données restent dans leur base de données respective.



Le consistent hashing permet ainsi de résoudre les principales limites du hachage classique en réduisant drastiquement le nombre de données redistribuées lors des changements de topologie (ajout ou retrait de base de données).

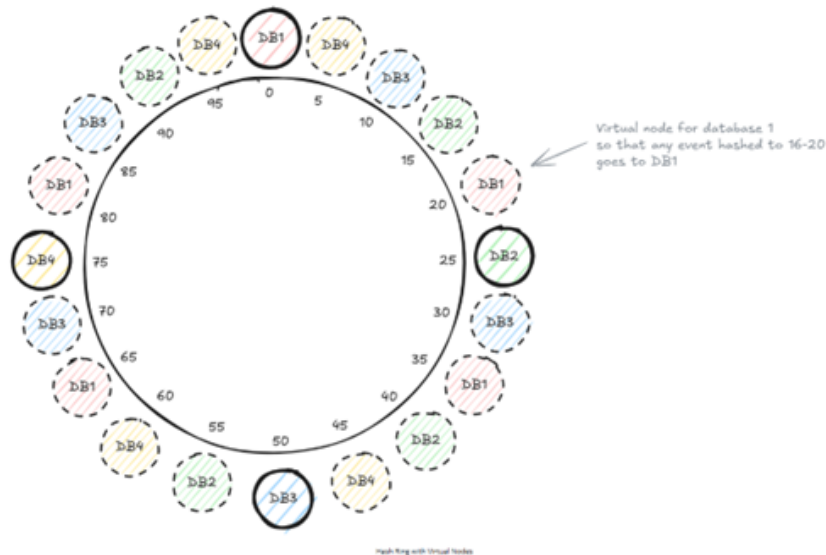
Cependant, il reste encore un problème. En effet, dans notre exemple ci-dessus où nous avons supprimé la base de données 2, nous avons dû déplacer toutes les données stockées dans la base de données 2 vers la base de données 3, ce qui peut entraîner une surcharge locale. Notre objectif serait de répartir de manière équitable les données lors de la suppression d'une base de données.

Optimisation : Nœuds Virtuels

Pour résoudre ce problème, une des solutions serait d'utiliser des « nœuds virtuels ». Au lieu de mettre chaque base de données en un seul point de sur l'anneau, nous la divisons en plusieurs points en hachant différentes variantes du nom de la base de données.

Par exemple :

« DB1 » sera divisé en « DB1-vn1 », « DB1-vn2 », « DB1-vn3 » et « DB1-vn4 » qui seront respectivement aux positions 0, 20, 35, 65, 80. C'est le même principe pour les autres bases de données.



Nous pouvons observer sur la figure ci-dessus que les nœuds virtuelles sont mélangés autour de l'anneau. Supposons que la bases de données numéro 2 rencontre une défaillance :

- Les données mappées sur « DB2-vn1 » seront stockées dans la base de données 1
- Les données mappées sur « DB2-vn2 » seront stockées dans la base de données 3
- Les données mappées sur « DB2-vn3 » seront stockées dans la base de données 4
- Et ainsi de suite

La charge de la base de données supprimé est donc répartie de manière beaucoup plus homogène dans toute les bases de données actives.

2.3 Conclusion sur les méthodes de répartitions des données

Les méthodes de répartition des données sont donc indispensables dans les systèmes distribués. En effet, elles déterminent la manière dont les données sont réparties entre les bases de données. Le hachage classique est une solution simple, mais peu adaptée lors de l'ajout ou du retrait d'une ou plusieurs bases de données, en raison des redistributions massives qu'il engendre. Le consistant hashing permet de corriger ces limitations en réduisant les déplacements de données. Une des optimisations du consistant hashing repose sur l'utilisation de nœuds virtuels, permettant d'améliorer l'équilibrage de charge entre les nœuds lors du retrait d'une base de données.

3 Approche naïve : une DHT non structurée inspirée de Gnutella

Cette première implémentation ne repose pas encore sur une vraie DHT structurée au sens de *Chord* ou *Pastry*. Elle correspond plutôt à un réseau pair-à-pair non structuré, inspiré de *Gnutella 0.4*, dans lequel chaque nœud participe à la fois au stockage, au routage et aux échanges réseau. Cette approche a constitué une étape utile pour valider les mécanismes de communication, de découverte et de tolérance aux déconnexions avant l'introduction d'un anneau de hachage cohérent.

3.1 Positionnement général et lien avec Gnutella

Le système suit un modèle *pure P2P* : il n'existe aucune séparation entre clients et serveurs. Chaque processus est un *servent* (*server* + *client*) capable de recevoir des connexions entrantes et d'initier lui-même des connexions sortantes.

Aspect	Implémentation réalisée	Gnutella 0.4
Topologie	Maillage non structuré	Maillage non structuré
Routage	Inondation limitée par TTL	Inondation limitée par TTL
Découverte	PING/PONG	PING/PONG
Recherche	GET	<i>Query</i>
Réponse	REPLY direct	<i>QueryHit</i>
Centralisation	Aucune	Aucune

TABLE 1 – Résumé comparatif entre l'approche naïve développée et les principes de *Gnutella 0.4*.

Cette parenté avec *Gnutella* n'est pas seulement conceptuelle. Elle décrit bien l'intuition de départ : construire d'abord un système entièrement décentralisé, simple à expérimenter, puis observer empiriquement ses limites avant de passer à une DHT structurée.

3.2 Problème initial : l'absence de vue globale

Le premier problème rencontré ne vient pas du transport réseau, mais du placement des données. Dans une topologie trop clairsemée, chaque pair ne connaît qu'un sous-ensemble des autres nœuds. Or si le choix du propriétaire d'une clé dépend de la vue locale du pair qui effectue le calcul, deux machines peuvent aboutir à des décisions différentes pour la même donnée.

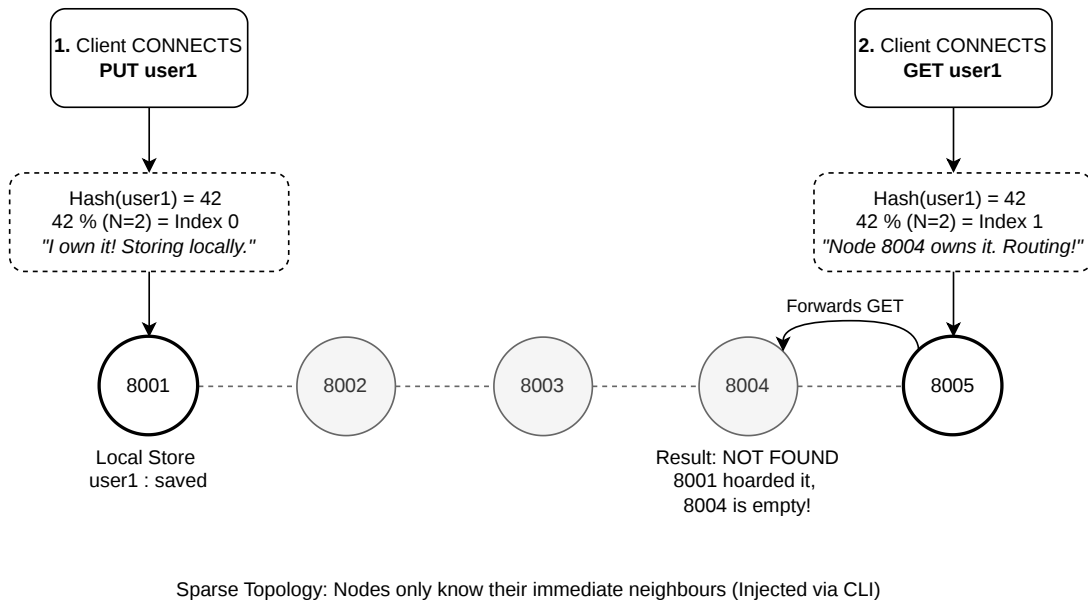


FIGURE 12 – Problème de vue globale dans une topologie clairsemée. Le nœud 8001 stocke localement `user1` après avoir calculé $\text{Hash}(\text{user1}) = 42$ avec sa propre vision du système. Plus tard, le nœud 8005 effectue un GET pour la même clé, mais avec une vue réseau différente il conclut que le propriétaire devrait être 8004 et relaie donc la requête dans une mauvaise direction. L'exemple montre qu'en l'absence d'un espace d'adressage partagé et stable, deux nœuds peuvent prendre des décisions incompatibles pour une même clé.

Ce constat a motivé un premier choix d'implémentation : tester explicitement une **structure en mesh** afin d'améliorer la cohérence de la vue réseau. Les schémas présentés dans cette section ne sont donc pas seulement illustratifs ; ils correspondent à des scénarios de test que nous avons utilisés pour vérifier le comportement du système lorsque tous les nœuds se connaissent, puis lorsqu'un nœud disparaît.

On peut formaliser cette intuition simplement. Si l'on note V l'ensemble des nœuds actifs et E l'ensemble des liens de voisinage, alors un mesh complet correspond au graphe non orienté

$$G = (V, E), \quad E = \{\{u, v\} \mid u, v \in V, u \neq v\}.$$

Pour $|V| = N$, le nombre total d'arêtes vaut alors

$$|E| = \binom{N}{2} = \frac{N(N-1)}{2}.$$

Chaque nœud a donc un degré exactement égal à

$$\deg(v) = N - 1.$$

Cette propriété explique pourquoi la vue locale devient beaucoup plus cohérente : en régime stable, chaque pair voit directement tous les autres.

3.3 Passage à un mesh : ce que cela corrige, et ce que cela casse

Dans une topologie en maillage complet, chaque nœud possède a priori la même vue des participants actifs. Cette organisation réduit donc fortement le problème de vue globale : tant que le réseau reste stable, les pairs appliquent le même calcul de placement et convergent plus facilement vers le même propriétaire logique. Elle corrige aussi partiellement le problème du modulo naïf, car la cohérence de $\mathbb{N}$ est meilleure tant que tous les pairs partagent la même vision du système.

En revanche, cette amélioration reste fragile. Dès qu'un nœud rejoint ou quitte le réseau, la valeur de N change et la fonction de placement $\text{hash}(\text{key}) \% N$ peut désigner un autre propriétaire. Le maillage complet réduit donc l'incohérence à l'instant donné, mais ne supprime pas la dépendance structurelle au nombre de nœuds.

Cette fragilité se montre immédiatement par un petit raisonnement. Pour une clé de hachage h , le propriétaire logique dans le schéma naïf est

$$\text{owner}_N(h) = h \bmod N.$$

Après le départ d'un nœud, le système calcule au contraire

$$\text{owner}_{N-1}(h) = h \bmod (N - 1).$$

Les deux valeurs ne coïncident pas en général. Il suffit d'un contre-exemple pour le voir : avec $h = 10$ et $N = 4$, on obtient

$$10 \bmod 4 = 2, \quad 10 \bmod 3 = 1.$$

Le propriétaire logique change donc alors même que la donnée n'a pas physiquement bougé. Plus généralement, l'ensemble des clés qui restent stables après le passage de N à $N - 1$ est

$$S = \{h \in \mathbb{N} \mid h \bmod N = h \bmod (N - 1)\},$$

et cet ensemble est strictement plus petit que l'ensemble de toutes les clés ; il n'existe donc aucune garantie globale de stabilité sous churn.

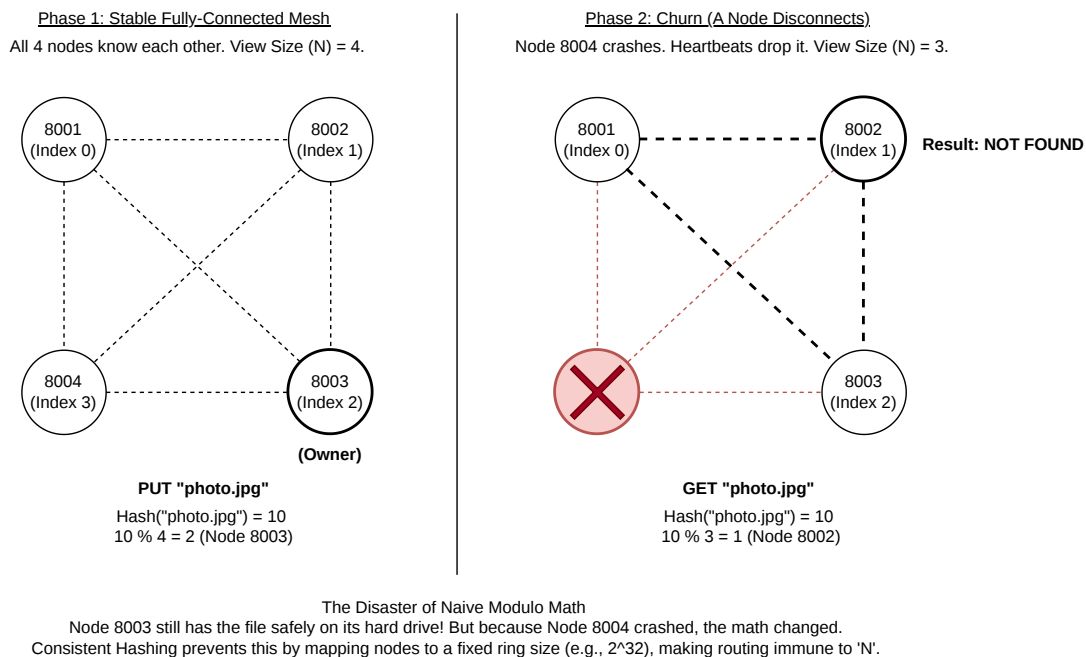


FIGURE 13 – Impact du *churn* sur un placement basé sur $\text{hash}(\text{key}) \% N$. À gauche, avec quatre nœuds connectés en mesh, la clé `photo.jpg` est associée au nœud 8003, qui la stocke localement. À droite, après la panne du nœud 8004, la taille de la vue passe de 4 à 3 ; le même calcul désigne alors 8002 comme propriétaire logique. Le mesh améliore donc la cohérence de la vue tant que le système est stable, mais il ne protège pas contre l'instabilité du placement lorsque N varie.

Autrement dit, la structure en mesh règle en grande partie le problème de vue globale en régime stable, mais elle fait aussi apparaître plus clairement le vrai défaut du schéma naïf : la dépendance du placement au cardinal du réseau. Les tests de churn ont précisément servi à mettre ce point en évidence.

Du point de vue de la complexité, le mesh complet présente aussi un coût structurel élevé. Comme chaque nœud maintient $N - 1$ voisins, la mémoire consacrée aux tables de voisinage est en $\Theta(N)$ par pair, soit $\Theta(N^2)$ à l'échelle du système. De même, un heartbeat envoyé à tous les voisins toutes les $\Delta = 5$ secondes produit environ

$$N(N - 1)$$

messages de contrôle par période si l'on compte chaque émission directionnelle. Le coût de maintenance croît donc quadratiquement avec la taille du réseau.

3.4 Le flooding comme correction partielle

Une fois ce problème observé, la question n'était plus seulement « où stocker ? », mais aussi « comment retrouver une donnée quand le calcul local du propriétaire devient faux ou incomplet ? ». C'est là que l'inondation limitée joue un rôle utile.

Dans cette version, lorsqu'un nœud reçoit une requête GET et qu'il ne possède pas localement la donnée recherchée, il la retransmet à l'ensemble de ses voisins connus. Le mécanisme est donc une **inondation contrôlée**. Ce n'est pas une solution parfaite au problème de placement, mais c'est une **solution de rattrapage** : même si un nœud se trompe sur le propriétaire logique, la requête peut encore atteindre le bon détenteur effectif de la donnée à force de propagation dans le mesh.

- **Diffusion.** Une requête est rediffusée de pair en pair jusqu'à trouver un propriétaire potentiel.
- **TTL.** Un plafond `MAX_HOPS = 10` limite le nombre de sauts pour éviter les boucles infinies et contenir le trafic.
- **Élimination des doublons.** Le `PeerRegistry` mémorise les identifiants déjà vus, par exemple sous la forme `Type|Origin|Seq`, afin d'ignorer immédiatement les messages dupliqués.

On peut borner ce coût de recherche. Dans un graphe de degré moyen d , une inondation de profondeur maximale T génère, dans le pire cas sans élimination efficace des doublons, un nombre de transmissions majoré par une série géométrique :

$$1 + d + d^2 + \dots + d^T = \sum_{i=0}^T d^i.$$

Si $d > 1$, cette somme vaut

$$\sum_{i=0}^T d^i = \frac{d^{T+1} - 1}{d - 1},$$

soit un coût exponentiel en T . Dans un mesh complet, on peut prendre $d = N - 1$, ce qui donne une borne très pessimiste mais révélatrice : la redondance potentielle devient rapidement prohibitive quand N grandit.

Grâce au cache de messages déjà vus, l'implémentation réelle est heureusement plus proche d'un parcours borné des arêtes accessibles que d'une explosion purement exponentielle. Dans ce cas, une requête GET touche au plus chaque lien utile un nombre constant de fois, ce qui ramène l'ordre de grandeur à $O(|E|)$. Pour un mesh complet, cela reste

$$O(|E|) = O\left(\frac{N(N - 1)}{2}\right) = O(N^2).$$

Même avec ce filtrage, le pire cas reste donc quadratique en nombre de nœuds. On peut l'écrire plus explicitement dans le cas d'un mesh complet. Comme $|E| = \frac{N(N-1)}{2}$, une recherche GET qui doit potentiellement explorer tout le réseau avant de conclure à un échec, ou avant d'atteindre le seul nœud possédant la donnée en dernier, effectue dans le pire cas un nombre de transmissions proportionnel à

$$T_{\text{worst}}(N) \approx 2|E| = N(N - 1).$$

Le facteur 2 vient du fait qu'un lien bidirectionnel peut être sollicité dans les deux sens au cours de la propagation et des relais. On obtient donc

$$T_{\text{worst}}(N) \in \Theta(N^2).$$

Autrement dit, doubler le nombre de nœuds multiplie approximativement par quatre le coût maximal d'une recherche exhaustive, ce qui confirme que cette stratégie ne passe pas bien à l'échelle.

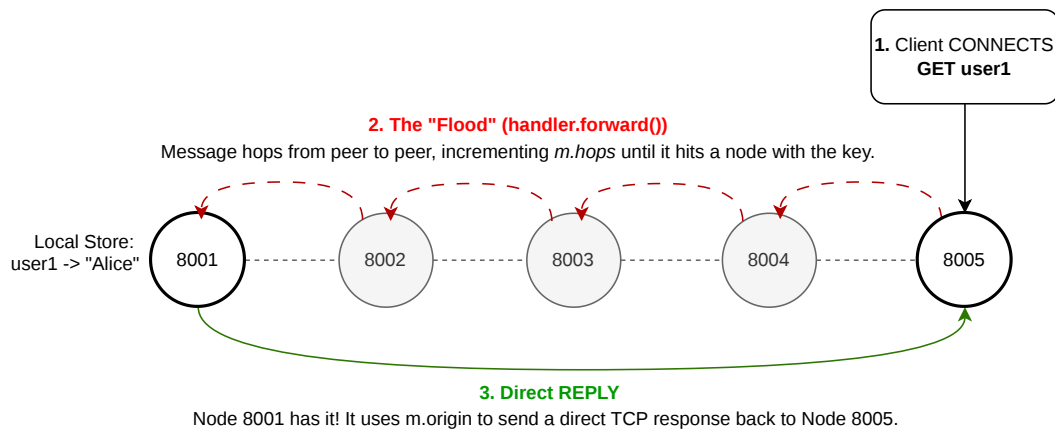


FIGURE 14 – Recherche par inondation dans l'approche naïve. Le nœud 8005 émet une requête GET `user1` ; celle-ci est relayée de proche en proche par les pairs intermédiaires, avec incrément du compteur de sauts, jusqu'au nœud 8001 qui possède la donnée dans son stockage local. Dans ce sens, le flooding corrige partiellement les erreurs de placement ou de vue locale : même si le demandeur ne connaît pas le bon propriétaire, la propagation augmente ses chances d'atteindre le nœud qui détient effectivement la donnée.

Ces bornes montrent bien l'ambivalence du flooding dans cette architecture naïve. D'un côté, il constitue un mécanisme de rattrapage robuste : si le réseau reste connexe, qu'aucun lien critique n'échoue pendant la recherche, et que le TTL est au moins égal au diamètre D du graphe, alors une requête émise depuis un nœud source finit nécessairement par atteindre tout nœud cible détenant la donnée. Dans un mesh complet, le diamètre vaut

$$D = 1,$$

ce qui explique pourquoi le flooding y est particulièrement efficace : tant que le système reste stable, la découverte est presque immédiate.

D'un autre côté, cette propriété se paie cher. Le flooding ne résout le problème que partiellement : il compense une mauvaise localisation par davantage de trafic réseau. Le système devient donc plus tolérant aux incohérences, mais au prix d'un coût en messages qui croît vite avec la taille du réseau. Cette tension entre *robustesse de découverte* et *coût quadratique ou pire* résume précisément la limite de l'approche naïve.

3.5 Réponse directe, protocole et concurrence

Un point positif de l'implémentation est que la phase de réponse est plus efficace que la phase de recherche. Lorsqu'un nœud trouve la clé demandée, il n'inonde pas le réseau avec la réponse. Il exploite le champ `origin` du message pour renvoyer directement un REPLY au demandeur au moyen d'une connexion TCP dédiée. On obtient ainsi un schéma classique : découverte diffuse, retour ciblé. Toutes les communications utilisent TCP afin d'assurer une livraison fiable des messages. Les objets Java sont sérialisés par `ObjectOutputStream` et désérialisés via `ObjectInputStream`. Afin de réduire les risques liés à la désérialisation, le `ConnectionHandler` applique un filtre minimal qui n'accepte que des classes issues d'espaces sûrs, par exemple `dht.*` et `java.lang.*`.

Le traitement des connexions suit un modèle *thread-per-connection* :

- chaque socket accepté par `NodeServer` est confié à un `ConnectionHandler` exécuté dans son propre thread ;

- le service `HeartbeatService` tourne en arrière-plan comme thread démon afin de vérifier périodiquement la vivacité des voisins ;
- les structures partagées sont rendues sûres vis-à-vis de la concurrence grâce à une `ConcurrentHashMap` pour le stockage local et à une `CopyOnWriteArrayList` pour la liste des pairs.

Cette organisation permet de gérer plusieurs pairs simultanément sans bloquer la logique applicative principale, tout en restant relativement simple à implémenter.

3.6 Découverte et maintien de la vivacité

Le maintien du réseau repose sur une gestion active de la présence des pairs.

- **Propagation des arrivées et départs.** Lorsqu'un nœud rejoint le système, il annonce sa présence et ses voisins relaient l'information afin d'améliorer progressivement la convergence de la vue réseau.
- **Heartbeat.** Toutes les 5 secondes, `HeartbeatService` envoie des messages `PING` aux pairs connus.
- **Détection de panne.** Un pair est retiré du `PeerRegistry` s'il ne répond plus, ou s'il reste silencieux pendant plus de 15 secondes (`PEER_TIMEOUT_MS`).

Ces mécanismes rendent le réseau dynamique et autonome, mais ils confirment également qu'une topologie non structurée reste sensible au *churn*. La découverte et le mesh améliorent la connectivité ; ils ne suffisent pas à produire un placement stable des clés.

3.7 Usages réels et intérêt pratique

Il serait toutefois réducteur de conclure que ce type d'architecture n'a aucun intérêt pratique. Sous sa forme brute, un réseau entièrement de type Gnutella passe mal à l'échelle pour un stockage distribué global, précisément à cause du coût du flooding et de l'instabilité du placement. En revanche, ses idées restent très utiles dans des contextes où la simplicité, la robustesse et la découverte rapide priment sur l'optimalité asymptotique.

On retrouve aujourd'hui ces idées dans des briques très concrètes.

Sur un réseau local, **mDNS** (*multicast DNS*) est un mécanisme de découverte sans infrastructure (<https://datatracker.ietf.org/doc/html/rfc6762>). Son rôle est simple : permettre à des machines présentes sur le même réseau de s'annoncer mutuellement et de découvrir rapidement leurs voisins sans passer par un serveur central. L'idée est très proche de notre approche naïve : on privilégie la simplicité, la visibilité rapide et l'absence de coordination centrale.

Dans l'écosystème **libp2p**, on retrouve également cette logique pour la découverte locale des pairs (<https://libp2p.io/docs/mdns/>). Un nœud peut ainsi détecter d'autres participants proches et établir rapidement ses premières connexions. À un autre niveau, les protocoles *Floodsub* puis surtout *Gossipsub* servent à diffuser des messages dans le réseau pair-à-pair (<https://libp2p.io/docs/pubsub/>). Ils reprennent l'intuition du flooding, mais sous une forme mieux contrôlée afin de limiter le trafic inutile.

Dans les **blockchains modernes**, la couche réseau pair-à-pair conserve elle aussi cette idée de propagation rapide (<https://ethereum.org/developers/docs/networking-layer/>). Concrètement, les nœuds commencent par découvrir d'autres pairs, puis utilisent un mécanisme de type gossip pour transmettre transactions et blocs à travers le réseau. L'objectif n'est pas de stocker les données via une DHT naïve, mais de propager vite et sans point central une information qui doit atteindre un grand nombre de participants.

Autrement dit, l'approche naïve inspirée de Gnutella reste pertinente comme brique de base pour des tâches bien ciblées : trouver rapidement des voisins, diffuser une information à grande vitesse, ou maintenir une connectivité décentralisée sans coordinateur central. Ce ne sont pas les mêmes objectifs qu'une DHT structurée ; c'est précisément pourquoi, dans les systèmes réels, ces mécanismes coexistent souvent avec des structures plus élaborées au lieu de les remplacer entièrement.

3.8 Bilan

Cette première version a permis de valider plusieurs briques essentielles : communication TCP fiable, sérialisation des messages, gestion des connexions concurrentes, découverte des pairs, surveillance de la vivacité et recherche distribuée. Surtout, elle nous a permis de tester deux idées importantes : d'une part, une structure en mesh améliore nettement la cohérence de la vue réseau ; d'autre part, le flooding permet de retrouver une donnée même lorsque le calcul local du propriétaire devient faux.

Du point de vue des performances, cette architecture a toutefois un comportement très contrasté. Dans le meilleur cas, si la donnée est locale ou si le bon pair est immédiatement accessible, une requête GET se résout en pratique en $O(1)$ sauts. Dans le pire cas, au contraire, la recherche doit explorer une grande partie du réseau ; sur un mesh complet, son coût atteint alors $\Theta(N^2)$ en nombre de transmissions. De la même manière, la topologie complète offre un diamètre minimal égal à 1, ce qui maximise la rapidité de découverte, mais son coût structurel reste quadratique : chaque pair maintient $N - 1$ voisins, soit une mémoire en $\Theta(N)$ localement et en $\Theta(N^2)$ à l'échelle du système.

Le rôle de la topologie est donc central. Une topologie clairsemée réduit les coûts de maintenance, mais elle produit des vues locales divergentes et donc des décisions incompatibles sur le placement des clés. À l'inverse, un mesh complet améliore fortement la cohérence de la vue réseau, mais il ne corrige pas le problème fondamental du placement par $\text{hash}(\text{key}) \% N$: dès que N varie, le propriétaire logique d'une clé peut changer. On obtient donc un compromis défavorable entre cohérence locale, coût de maintenance et stabilité globale du placement.

En revanche, pour le placement stable des données et le routage scalable, ces améliorations restent palliatives. Le mesh ne supprime pas la dépendance à N , et le flooding compense l'erreur par un surcoût en messages. Ces limites motivent directement l'introduction de vraies DHT structurées dans la suite du rapport. Des approches comme *Chord* ou *Pastry* remplacent la dépendance à N par un espace d'identifiants fixe et introduisent des mécanismes de routage déterministes, beaucoup mieux adaptés à un environnement distribué dynamique.

4 Chord

Chord est un protocole DHT (*Distributed Hash Table*) qui résout un problème unique : trouver rapidement le nœud responsable d'une clé dans un réseau pair-à-pair dynamique.

4.1 Principe de fonctionnement

Chaque nœud et chaque clé reçoivent un identifiant sur m bits (souvent via SHA-1), placé sur un anneau modulo 2^m . Une clé k est stockée sur son *successor* : le premier nœud rencontré dans le sens horaire dont l'identifiant est supérieur ou à k .

Cette règle donne trois propriétés utiles :

- **Répartition** : le *consistent hashing* distribue les clés de manière quasi uniforme.
- **Scalabilité** : l'ajout/retrait d'un nœud ne déplace qu'une fraction des clés.
- **Décentralisation** : aucun serveur central n'est requis.

4.2 Routage efficace : Finger Table

Pour éviter un parcours linéaire de l'anneau, chaque nœud maintient une **finger table** de m entrées. L'entrée i pointe vers le premier nœud joignable à partir de $n + 2^{i-1}$.

$$\text{finger}[i].\text{start} = (n + 2^{i-1}) \bmod 2^m$$

Cette structure permet des sauts exponentiels et un coût moyen de recherche en $O(\log N)$ messages.

4.3 Algorithmes essentiels

- 1) **Recherche de la clé** : `find_successor(id)`
 - Si `id` est entre le nœud courant et son *successor*, on retourne ce *successor*.
 - Sinon, on avance vers le *closest_preceding_node(id)* dans la finger table et on répète.
- 2) **Choix du prochain saut** : `closest_preceding_node(id)`
 - Parcourt de la finger table du plus grand saut au plus petit.
 - Retour du premier doigt situé strictement avant `id` sur l'anneau.
- 3) **Arrivée d'un nœud** : `join(n')`
 - Le nouveau nœud contacte un nœud connu `n'`.
 - Il initialise son *successor* via `find_successor` puis construit sa finger table.
- 4) **Stabilisation** : `stabilize()` et `notify()`
 - Exécution périodique pour corriger les pointeurs *successor/predecessor*.
 - Garantit la convergence vers un anneau cohérent après churn modéré.
- 5) **Tolérance aux pannes** : `successor list`
 - Chaque nœud conserve plusieurs successeurs de secours.
 - En cas de panne du successeur principal, le routage continue via le suivant vivant.

4.4 Complexité (résumé)

- **Lookup** : $O(\log N)$ messages.
- **Maintenance d'un join/leave** : surcoût logarithmique (dépend de la politique de mise à jour).
- **Mémoire par nœud** : $O(\log N)$ entrées de routage.

En pratique, Chord fournit une base simple et robuste pour le *lookup* distribué : routage logarithmique, maintenance locale périodique et bonne résilience aux variations du réseau.

5 Pastry

Pastry est une infrastructure distribuée de localisation d'objets et de routage destinée aux systèmes pair-à-pair (P2P) à grande échelle. Elle repose sur une table de hachage distribuée (DHT), comparable à Chord, et fonctionne au-dessus d'Internet via un réseau overlay. Les nœuds logiques de Pastry ne correspondent pas nécessairement à la topologie physique du réseau IP, mais forment une structure virtuelle organisée selon l'espace des identifiants.

L'objectif fondamental de Pastry est d'acheminer efficacement un message associé à une clé donnée vers le nœud dont l'identifiant est numériquement le plus proche de cette clé. À l'instar de Chord, Pastry résout le problème du *key-based routing*, mais adopte une approche différente fondée sur un routage par préfixes plutôt que sur une structure strictement circulaire.

5.1 Architecture générale

La conception de Pastry repose sur plusieurs principes structurants :

1. Décentralisation

Aucun nœud ne possède de rôle central. Chaque participant maintient localement ses structures de routage, ce qui supprime tout point unique de défaillance.

2. Scalabilité logarithmique

Le routage s'effectue en $O(\log N)$ sauts en moyenne, où N est le nombre de nœuds du système, ce qui garantit le passage à grande échelle.

3. Auto-organisation

Le protocole s'adapte dynamiquement aux arrivées et aux départs de nœuds (*churn*) par la mise à jour progressive des structures de routage.

4. Localité réseau

Lorsque plusieurs nœuds satisfont une contrainte de routage, Pastry privilégie celui qui est le plus proche selon une métrique réseau, réduisant ainsi la latence réelle des communications.

5.2 Identifiants et espace de clés

Chaque nœud Pastry possède un identifiant unique, appelé *nodeId*, généralement codé sur 128 bits et dérivé d'une fonction de hachage cryptographique. De même, chaque message ou clé à router reçoit un identifiant correspondant dans le même espace.

L'espace d'identifiants est organisé en base 2^b , où b est un paramètre de configuration influençant la taille des tables de routage. Le routage s'effectue en recherchant le nœud actif dont le *nodeId* est numériquement le plus proche de l'identifiant cible.

Contrairement à Chord, qui dispose d'un anneau strictement circulaire, Pastry utilise un routage par préfixes, où la distance est mesurée par la longueur du préfixe commun entre identifiants.

5.3 Structures de routage

Chaque nœud Pastry maintient trois structures principales : la *Routing Table*, le *Leaf Set* et le *Neighbor Set*.

5.3.1 1. Routing Table

Organisée par préfixes d'identifiants, elle contient des pointeurs vers des nœuds partageant des préfixes communs avec le *nodeId* courant. Cette structure permet de faire des sauts logarithmiques vers la destination, rapprochant le message à chaque étape.

Cette structure est analogue aux *Finger Tables* de Chord, mais basée sur des préfixes plutôt que sur des puissances de 2.

5.3.2 2. Leaf Set (ensemble des feuilles)

Il regroupe les nœuds immédiatement supérieurs et inférieurs au *nodeId*. Il assure le routage correct pour les clés proches et fournit des alternatives en cas de défaillance de certains nœuds.

5.3.3 3. Neighbor Set (ensemble des voisins)

Il contient des nœuds physiquement proches selon la topologie réseau, indépendamment des identifiants. Il est utilisé pour optimiser la latence des communications et améliorer la localité réelle du routage.

5.4 Processus de routage

Le routage dans Pastry s'effectue en rapprochant progressivement le message de son *nodeId* cible grâce aux structures précédentes :

1. **Vérification du Leaf Set** : si la clé cible se trouve dans l'intervalle couvert, le message est envoyé au nœud le plus proche numériquement.

2. **Utilisation de la Routing Table** : si la clé n'est pas dans le Leaf Set, le message est transmis à un nœud partageant un préfixe plus long avec la clé.
3. **Sélection par proximité physique** : si aucune entrée n'est disponible ou valide, un nœud du Leaf Set numériquement plus proche est choisi, en prenant en compte la distance réseau.

À chaque étape, le message progresse vers un nœud plus proche de la destination, ce qui garantit la convergence du routage en $O(\log N)$ sauts en moyenne pour N nœuds actifs.

5.5 Algorithmes des nœuds et maintenance

Chaque nœud Pastry exécute des algorithmes pour intégrer de nouveaux nœuds, maintenir les structures de routage et gérer les défaillances.

1. **Intégration d'un nouveau nœud (*join*)**
 - Un nœud entrant contacte un nœud existant pour s'intégrer au réseau.
 - Les structures de routage (*Routing Table*, *Leaf Set*, *Neighbor Set*) sont initialisées progressivement grâce aux informations des nœuds voisins.
2. **Maintenance dynamique (*stabilize* et mise à jour du routage)**
 - Les entrées de la *Routing Table* sont périodiquement mises à jour pour refléter les départs ou arrivées de nœuds.
 - Le *Leaf Set* et le *Neighbor Set* sont également ajustés pour maintenir la cohérence et la résilience locale.
3. **Tolérance aux pannes**
 - En cas de défaillance d'un nœud, les messages sont redirigés vers le nœud le plus proche dans le *Leaf Set* ou via la *Routing Table*.
 - Cette redondance garantit que le routage reste correct et efficace même avec un nombre important de nœuds défaillants.

6 Architecture du système et cycle de vie d'une requête

6.1 Choix architecturaux et problématique

Dans le cadre de notre projet, nous avons choisi d'implémenter une architecture de stockage distribuée reposant sur des protocoles de type Distributed Hash Table (DHT), et plus précisément sur Chord et Pastry. Ces deux protocoles partagent un objectif commun : organiser un ensemble de nœuds distribués de manière totalement décentralisée afin d'assurer le stockage et la localisation efficace de données dans un espace d'identifiants partagé.

Avant d'implémenter ces protocoles, il est essentiel de comprendre précisément l'architecture que nous allons construire ainsi que le protocole applicatif qui s'appuiera dessus. Autrement dit, nous devons être capables de répondre aux questions suivantes :

- À quel nœud une requête est-elle envoyée initialement ?
- Comment est déterminé le nœud responsable d'une donnée ?
- Comment les nœuds communiquent-ils entre eux ?
- Que se passe-t-il concrètement lors d'une opération élémentaire telle que PUT ou GET ?
- Quel est le cycle de vie complet d'une requête ?

Dans cette première partie, nous détaillons l'architecture retenue pour Chord, en analysant précisément la circulation des messages, la responsabilité des nœuds et le traitement des requêtes. La même démarche sera ensuite appliquée à Pastry.

6.2 Architecture générale basée sur Chord

6.2.1 Couche Overlay et Couche applicative

Il est essentiel, dans notre implémentation, de distinguer clairement deux niveaux fonctionnels qui coexistent au sein de chaque nœud.

Le premier niveau correspond à la couche overlay, c'est-à-dire le protocole Chord lui-même. Cette couche est responsable de l'organisation logique du réseau en anneau, du maintien des pointeurs (successeur, prédécesseur, finger table) et du routage des messages. Son rôle est exclusivement de répondre à la question suivante : quel nœud est responsable d'un identifiant donné ? Autrement dit, Chord assure la localisation distribuée des clés.

Le second niveau correspond à la couche applicative, que nous implémentons au-dessus de Chord. Cette couche gère concrètement le stockage des paires (clé, valeur) sur le nœud responsable. Elle implémente les opérations élémentaires telles que PUT, GET ou DELETE, et décide éventuellement des mécanismes de réplication ou de gestion de la cohérence.

Ainsi, Chord ne stocke pas directement les données : il fournit un mécanisme de routage déterministe permettant d'identifier le bon nœud. La gestion effective des données repose sur la couche applicative locale de chaque nœud.

6.2.2 Cycle de vie d'une requête

Requête PUT

Lorsqu'un client souhaite insérer une donnée via PUT(key, value), la requête est envoyée à un nœud arbitraire du réseau. Ce nœud devient le point d'entrée du système, mais il n'est pas nécessairement responsable de la clé.

Le nœud calcule d'abord l'identifiant associé à la clé en appliquant la fonction de hachage. L'identifiant obtenu est positionné dans le même espace circulaire que les nœuds.

Le nœud exécute alors la procédure `find_successor(id)`. Si l'identifiant appartient à l'intervalle compris entre le nœud courant et son successeur, alors ce successeur est responsable. Sinon, le nœud consulte sa finger table pour déterminer le `closest preceding node`, c'est-à-dire le nœud le plus proche

précédant la cible. La requête est alors transmise à ce nœud. Ce processus se répète récursivement jusqu'à atteindre le nœud responsable.

Une fois le nœud responsable localisé, la paire (clé, valeur) est stockée localement. Pour assurer une tolérance aux pannes, cette donnée peut être répliquée sur plusieurs successeurs afin d'assurer une redondance. Une réponse de confirmation est ensuite renvoyée au client.

Requête GET

Le traitement d'une requête GET(key) suit exactement la même logique de routage que PUT. La différence réside uniquement dans l'opération finale.

Après avoir localisé le nœud responsable via `find_successor`, celui-ci consulte son stockage local pour récupérer la valeur associée à la clé. Si la clé est présente, la valeur est renvoyée au client. Sinon, une réponse d'absence est transmise au client.

La latence globale d'une requête GET dépend principalement du nombre de sauts nécessaires pour atteindre le nœud responsable, ce qui reste logarithmique.

Requête DELETE

Le traitement d'une requête DELETE(key) suit le même mécanisme de localisation que les opérations PUT et GET. Le nœud qui reçoit la requête commence par calculer l'identifiant associé à la clé via la fonction de hachage, puis exécute la procédure `find_successor` afin de déterminer le nœud responsable. Une fois ce nœud atteint, la suppression est effectuée localement dans le stockage clé-valeur. Si un mécanisme de réplication est implémenté, la suppression doit également être propagée aux nœuds répliqués afin de maintenir la cohérence du système. Une confirmation est ensuite renvoyée au client. Comme pour les autres opérations, la complexité de la localisation reste logarithmique, et la suppression ne nécessite aucune coordination globale en dehors du routage vers le nœud responsable.

6.2.3 Conclusion sur l'architecture Chord

L'architecture que nous mettons en place repose sur une séparation entre le mécanisme de localisation distribué et la couche applicative de stockage. Cette structuration nous permet de construire un système scalable et entièrement décentralisé, dans lequel chaque nœud participe à la fois au routage et au stockage des données.

Cependant, si l'overlay garantit une localisation efficace des clés, la robustesse réelle du système dépendra des choix que nous ferons au niveau applicatif, notamment en matière de réplication, de gestion des pannes et de cohérence. La stabilité dynamique de l'anneau, en particulier en présence de churn, influencera directement les performances et la fiabilité du système.

Nous considérons ainsi cette architecture comme une base solide et extensible, dont le comportement effectif dépendra de la rigueur de notre implémentation et des mécanismes complémentaires que nous déciderons d'y intégrer.

6.3 Architecture générale basée sur Pastry

6.3.1 Couche overlay et couche applicative dans Pastry

Comme pour Chord, il est essentiel de distinguer deux niveaux fonctionnels.

La couche overlay correspond ici au protocole Pastry lui-même. Elle est responsable de la gestion des identifiants, du maintien des trois structures de routage et du routage progressif des messages vers le nœud cible.

Le routage s'effectue en augmentant à chaque étape la longueur du préfixe commun entre l'identifiant du nœud courant et celui de la clé recherchée. Si la clé se situe dans l'intervalle couvert par le Leaf Set, la résolution est immédiate. Sinon, la Routing Table est utilisée pour effectuer un saut vers un

nœud partageant un préfixe plus long.

La couche applicative, quant à elle, implémente le stockage clé-valeur sur le nœud responsable. Comme dans Chord, Pastry ne définit pas directement la politique de stockage : il fournit uniquement un mécanisme de localisation distribuée.

6.3.2 Cycle de vie d'une requête dans Pastry

Requête PUT

Lorsqu'un client envoie une requête PUT(key, value), celle-ci est transmise à un nœud d'entrée arbitraire du réseau.

Le nœud calcule l'identifiant de la clé via la fonction de hachage. Il compare ensuite cet identifiant à son Leaf Set. Si la clé se trouve dans l'intervalle couvert, il route directement vers le nœud numériquement le plus proche. Sinon, il consulte sa Routing Table afin de trouver un nœud partageant un préfixe plus long avec la clé.

Le message progresse ainsi de proche en proche. Chaque saut augmente la longueur du préfixe commun. Ce mécanisme garantit une complexité en $O(\log N)$.

Une fois le nœud le plus proche atteint, la paire (clé, valeur) est stockée localement. Comme pour Chord, un mécanisme de réplication peut être appliqué sur les nœuds du Leaf Set afin d'assurer la tolérance aux pannes.

Une confirmation est renvoyée au client.

Requête GET

La requête GET(key) suit exactement le même processus de routage que PUT.

Une fois le nœud responsable atteint, celui-ci consulte son stockage local. Si la clé existe, la valeur est renvoyée au client. Sinon, une réponse négative est envoyée.

La latence dépend du nombre de sauts nécessaires pour aligner progressivement les préfixes, ce qui reste en $O(\log N)$.

Requête DELETE

Le traitement de DELETE(key) reprend le même mécanisme de localisation.

Après calcul de l'identifiant, le message est routé selon la logique Leaf Set et Routing Table. Une fois le nœud cible atteint, la clé est supprimée du stockage local. Si une réplication existe, la suppression est propagée aux réplicas du Leaf Set.

6.3.3 Conclusion sur l'architecture Pastry

L'architecture Pastry repose sur un routage par préfixes qui permet d'atteindre efficacement le nœud le plus proche d'une clé tout en tenant compte de la proximité réseau. Cette approche garantit une complexité logarithmique et peut améliorer la latence réelle des communications.

Cependant, la maintenance simultanée de plusieurs structures de routage (Routing Table, Leaf Set, Neighbor Set) rend le protocole plus complexe que Chord et plus sensible aux phénomènes de churn. L'efficacité globale du système dépendra donc directement de notre implémentation.